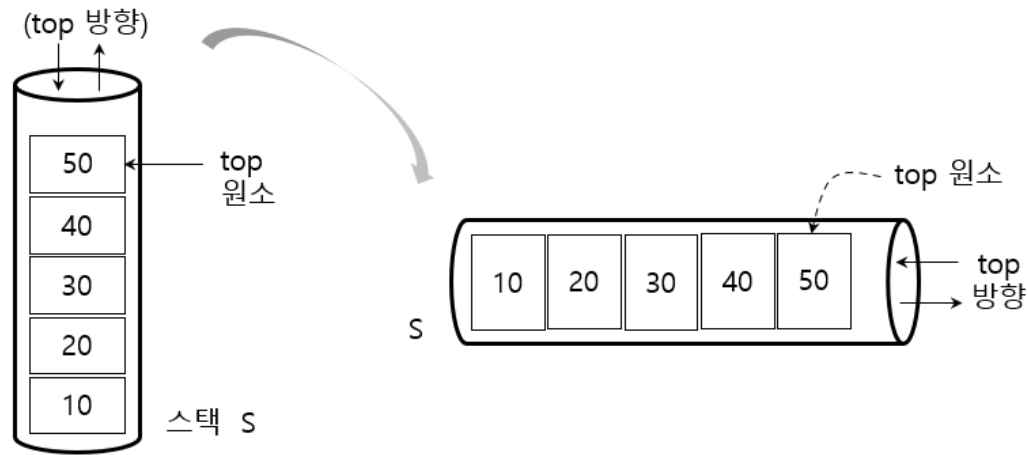


5.1 스택 : LIFO 리스트

- 스택(stack) : 객체들을 차곡차곡 쌓아 보관하는 것



실생활의 스택들

- 한쪽 끝에서만 원소들의 추가와 삭제가 수행되는 되는 구조, 순서리스트(저장순서)
- 한쪽 방향에서만 추가와 삭제를 수행할 수 있는 특별한 순서리스트
 - 원소들의 입출력 방향 : 탑(top) 방향
 - 가장 최근에 추가된 원소 : 탑(top) 원소(가장 먼저 삭제될 수 있는 원소)
- LIFO(Last In First Out) 리스트

5.2 스택 구현과 응용

(1) 순차 스택

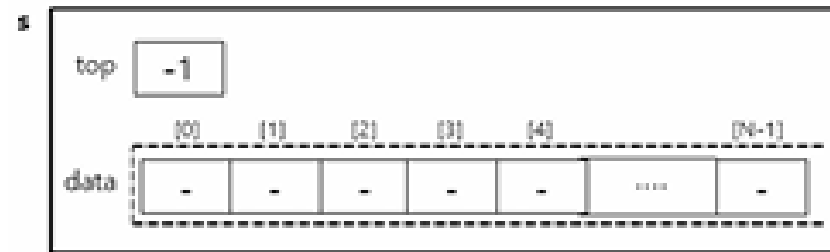
- 순차리스트로 구현한 스택 : 배열, lenth 변수 필요
- 장점 : 추가, 삭제를 수행해도 스택(순차리스트) 원소들의 이동이 없음

코드 5.1 순차스택 정의

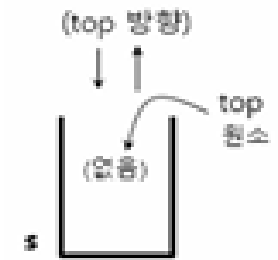
```
#define N 100

/* 정수스택 정의 */
typedef struct stack {
    int data[N] ;
    int top ;
} intStack;

intStack s;
s.top=-1; /* 스택 s 생성 */
```



(가) 스택의 배열 저장상태



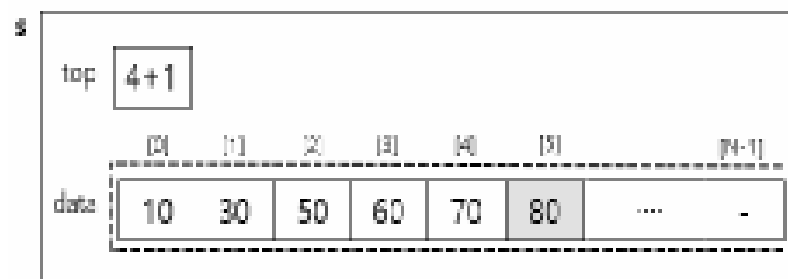
(나) 논리상태

그림 5.4 순차스택 s의 초기 상태

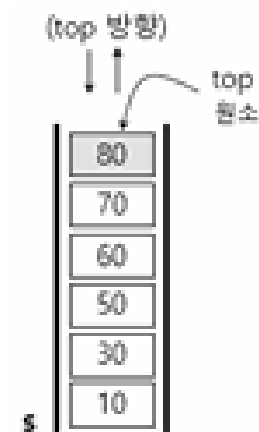
순차스택의 추가, 삭제연산

코드 5.2 순차스택의 추가

```
// 새 원소 x 추가 처리
if (s.top==N-1) overflow();
else {
    s.top = s.top+1;
    s.data[s.top] = x;
}
```



(가) 80 추가 후 스택 상태

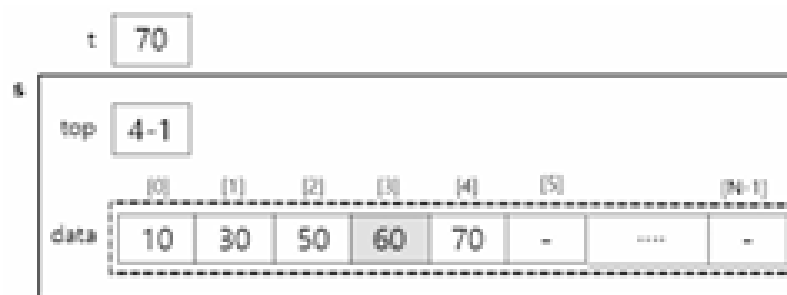


(나) 논리상태

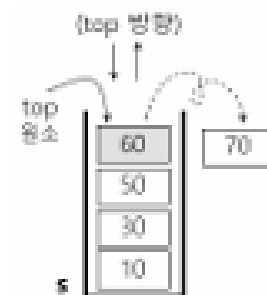
그림 5.6 순차스택의 추가연산 : 80 추가

코드 5.3 순차스택의 삭제

```
int t; // 삭제원소 저장
...
if (s.top == -1)
    underflow();
else {
    t = s.data[s.top];
    s.top = s.top-1;
}
```



(가) 탑원소(50) 삭제후의 스택 상태



(나) 논리상태

그림 5.7 순차스택의 삭제연산 : 70 삭제

코드 5.4 전역 순차스택과 처리함수

```
#define N 20
intStack s; // 전역 순차스택 정의
s.top = -1;

int pushS(int x) {
    if (s.top==N-1) return(0); // 추가실패
    s.top=s.top+1;
    s.data[s.top] = x;
    return(1); // 추가성공
}

int popS() {
    int t;
    if (s.top==-1) return(-1); // 삭제실패
    t=s.data[s.top];
    s.top=s.top-1;
    return(t); // 삭제원소 반환
}

void main(void) {
    ... // 스택 처리함수 호출
}
```

코드 5.5 지역 순차스택과 처리함수

```
#define N 20

int push(intStack *s, int x) {
    if (s->top==N-1) return(0); // 추가실패
    s->top=s->top+1;
    s->data[s->top]=x;
    return(1); // 추가성공
}

int pop(intStack *s) {
    int t;
    if (s->top==-1) return(-1); // 삭제실패
    t=s->data[s->top]; s->top=s->top-1;
    return(t); // 삭제원소 반환
}

void main(void) {
    intStack s; // 지역 순차스택 정의
    s.top=-1;
    ... // 스택 처리함수 호출
}
```

(3) 스택 응용

- 발생한 자료들의 역추적/역순처리, 피드-백 처리, 재귀적 처리 등이 필요한 응용에서 사용함
- 예
 - CPU의 인터럽트 처리, 운영체제의 함수호출관리, 수식변환과 계산, 재귀함수의 비재귀함수 변환 등
 - 미로문제, 이진트리의 순회(7장), 그래프의 깊이우선순회(9장) 등

(가) 수식표현 변환과 계산

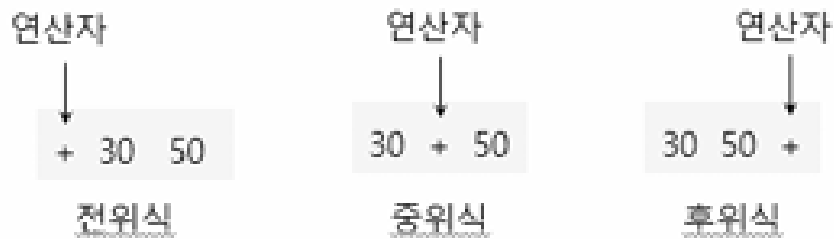


그림 5.10 수식 표현: 전위식, 중위식, 후위식

- 중위식 : 사람들의 수식표현 방법, 연산자 우선순위 개념이 있음
 - 단점 : 수식을 전부 읽은 후, 연산자 순위를 고려해서 연산을 처리해야 함
- 후위식 : 컴퓨터 내부의 수식표현 방법, 연산자우선순위 개념이 없음
 - 장점 : 수식을 읽어가면서 연산처리 가능, 괄호 표현이 없음

표 5.2 중위식과 후위식 예

중위식	후위식	결과
① $3+4$	$3\ 4\ +$	7
② $3*4+2$	$3\ 4\ *\ 2\ +$	14
③ $3+4*2$	$3\ 4\ 2\ *\ +$	11
④ $3+4*2+6/3$	$3\ 4\ 2\ *\ 6\ 3\ /\ +\ +$	13
⑤ $(3+4)*2$	$3\ 4\ +\ 2\ *$	14
⑥ $3+4*(2+6)/2$	$3\ 4\ 2\ 6\ +\ *\ 2\ /\ +$	19
⑦ $(A+B)*C+E/(F+A*D)+C$	$AB+C*EFAD*+/\ +C+$	-

표 5.3 연산자우선순위

연산자	우선순위	
	스택 안	스택 밖
(	0	8
)	7	7
~, !	6	6
*, /, %	5	5
+, -	4	4
<, <=, >, >=	3	3
&&	2	2
	1	1

■ 중위식의 후위식 변환 : 연산자 스택 사용

코드 5.8 중위식의 후위식 변환

```
/* Operand, Operator : 각각 피연산자, 연산자 집합 */
/* priority(op) : 연산자 op의 연산자 우선순위값 */
/* push(s, token); 스택 S에 token을 추가함 */
/* pop(s); 스택 S에서 top 원소를 삭제하여 반환함 */
/* print(token) : token을 출력함 */
/* discard(token) : token을 무시함 */
```

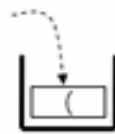
1. 연산자 스택 S를 준비한다;
2. while (중위식에서 토큰 token을 읽는다) {
 if (token in Operand) print(token);
 else if (token in Operator) {
 while (S is not empty) {
 if (priority(S.top) < priority(token)) break;
 print(pop(S));
 }
 push(s, token);
 }
 else if (token == '(') push(S, token);
 else if (token == ')')
 while (S.top != '(') print(pop(S));
 discard(pop(S));
 }
3. 스택 S의 모든 원소들을 출력한다;

■ 예:

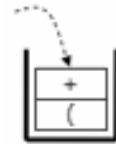
- 중위식 : $(A+B)*C+E/(F+A*D)+C$
- 후위식 : $AB+C*EFAD*+/+C+$

■ 연산자 스택을 이용한 중위식의 후위식 변환과정

- 중위식 : $(A+B)*C+E/(F+A*D)+C$
- 후위식 : $AB+C*EFAD*+/+C+$

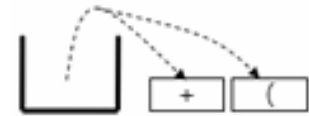


① $(A+B)*C+E/(F+A*D)+C$



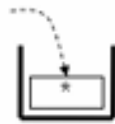
② $(A+B)*C+E/(F+A*D)+C$

출력: A



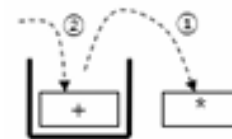
③ $(A+B)*C+E/(F+A*D)+C$

출력: AB+



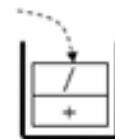
④ $(A+B)*C+E/(F+A*D)+C$

출력: AB+



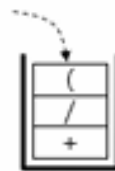
⑤ $(A+B)*C+E/(F+A*D)+C$

출력: AB+C*



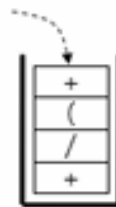
⑥ $(A+B)*C+E/(F+A*D)+C$

출력: AB+C*E



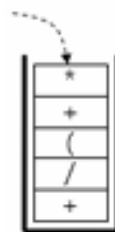
⑦ $(A+B)*C+E/(F+A*D)+C$

출력: AB+C*E



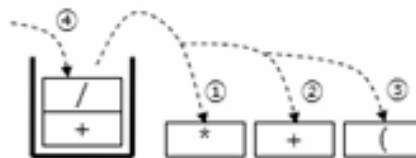
⑧ $(A+B)*C+E/(F+A*D)+C$

출력: AB+C*EF



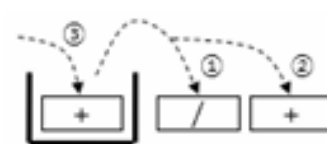
⑨ $(A+B)*C+E/(F+A*D)+C$

출력: AB+C*EFA



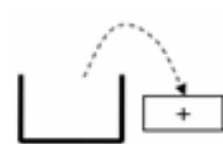
⑩ $(A+B)*C+E/(F+A*D)+C$

출력: AB+C*EFAD*+



⑪ $(A+B)*C+E/(F+A*D)+C$

출력: AB+C*EFAD*+/+



⑫ $(A+B)*C+E/(F+A*D)+C$

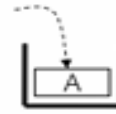
출력: AB+C*EFAD*+/+C+

■ 후위식의 계산 : 피연산자 스택 운영

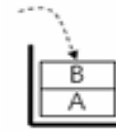
코드 5.9 후위식 계산

```
/* Operand, Operator : 각각 피연산자, 연산자 집합 */
/* calculate(op1, op, op2) : 연산자 op 연산식(op1 op op2)의 결과를 반환함 */
/* push(s, token); 스택 S에 token을 추가함 */
/* pop(s); 스택 S에서 top 원소를 삭제하여 반환함 */
```

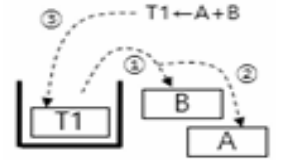
1. 피연산자 스택 S를 준비한다.
2. while (후위식에서 토큰 token을 읽는다) {
 - if (token in Operand) push(S, token);
 - else if (token in Operator) { /* token : 이항연산으로 가정함 */
 - operand2=pop(S); operand1 = pop(S);
 - result = calculate(operand1, token, operand2);
 - push(S, result);
3. 스택 S의 원소를 출력한다.



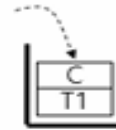
① AB+C*EFAD*+/+C+



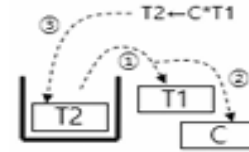
② AB+C*EFAD*+/+C+



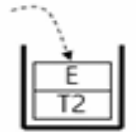
③ AB+C*EFAD*+/+C+



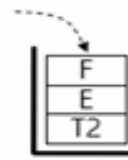
④ AB+C*EFAD*+/+C+



⑤ AB+C*EFAD*+/+C+



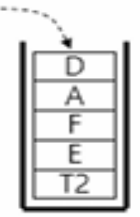
⑥ AB+C*EFAD*+/+C+



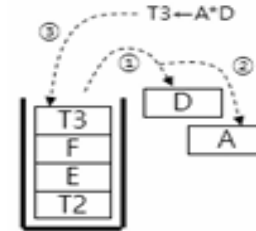
⑦ AB+C*EFAD*+/+C+



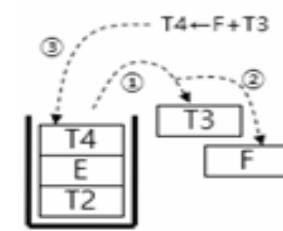
⑧ AB+C*EFAD*+/+C+



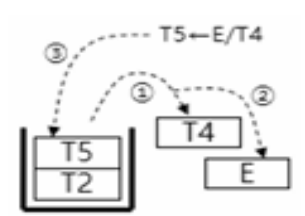
⑨ AB+C*EFAD*+/+C+



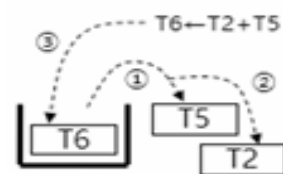
⑩ AB+C*EFAD*+/+C+



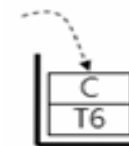
⑪ AB+C*EFAD*+/+C+



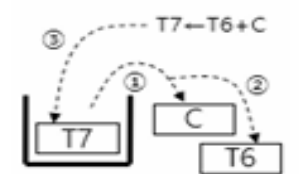
⑫ AB+C*EFAD*+/+C+



⑬ AB+C*EFAD*+/+C+



⑭ AB+C*EFAD*+/+C+



⑮ AB+C*EFAD*+/+C+

(나) 미로문제

- 2차원 배열 `maze[M][N]`으로 표현되는 미로배열의 입구에서 출구까지의 경로를 탐색하는 것
 - 미로배열: 0 - 통로, 1 - 벽, 입구 : `maze[0][0]`, 출구 : `maze[M-1][N-1]`
 - 이동 가능방향 : 8개 방향(북, 북동, 동, 남동, 남, 남서, 서, 북서방향)
 - 인접원소가 0이면 이동가능하고, 1이면 불가능함

→	0	1	0	0	0	1	1	0	0	0	1	1	1	1	1
	1	0	0	0	1	1	0	1	1	1	0	0	1	1	1
	0	1	1	0	0	0	0	1	1	1	1	0	0	1	1
	1	1	0	1	1	1	1	0	1	1	0	1	1	0	0
	1	1	0	1	0	0	1	0	1	1	1	1	1	1	1
	0	0	1	1	0	1	1	1	0	1	0	0	1	0	1
	0	1	1	1	1	0	0	1	1	1	1	1	1	1	1
	0	0	1	1	0	1	1	0	1	1	1	1	1	0	1
	1	1	0	0	0	1	1	0	1	1	0	0	0	0	0
	0	0	1	1	1	1	1	0	0	0	1	1	1	1	0
	0	1	0	0	1	1	1	1	0	1	1	1	1	1	0

(가) 미로문제의 2차원 배열 표현

	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
→	0	1	0	0	0	1	1	0	0	0	1	1	1	1	1
	1	1	0	0	0	1	1	0	1	1	1	0	0	1	1
	1	0	1	1	0	0	0	0	1	1	1	1	0	0	1
	1	1	1	0	1	1	1	1	0	1	1	0	1	1	0
	1	1	1	0	1	0	0	1	0	1	1	1	1	1	1
	1	0	0	1	1	0	1	1	1	0	1	0	0	1	0
	1	0	1	1	1	1	0	0	1	1	1	1	1	1	1
	1	0	0	1	1	0	1	1	0	1	1	1	1	0	1
	1	1	1	0	0	0	1	1	0	1	1	0	0	0	0
	1	0	0	1	1	1	1	1	0	0	0	1	1	1	0
	1	0	1	0	0	1	1	1	1	1	0	1	1	1	0
	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

(나) 미로문제의 2차원 배열 표현(수정)

● 미로탐색 과정

- 막다른 길에 도달하면, 직전 방문지점으로 돌아와서 탐색하지 않은 다른 인접점을 같은 방법으로 탐색함
- 고려 사항 : 미로탐색 이동방향 표현, 직전 방문점 정보, 직전 이동방향 정보, 인접점의 방문 여부 확인

■ 미로탐색 이동방향 표현

- 8개 이동방향 : 0~7의 정수 표현
- 편의상 이동방향은 북(0), 북동(1), 동(2), 남동(3), 남(4), 남서(5), 서(6), 북서방향(7) 순서로 선택함(시계방향)

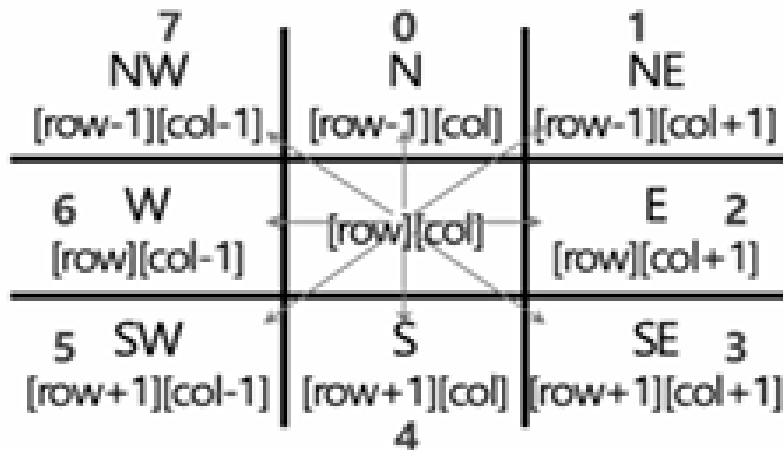


그림 5.14 미로탐색의 8개 이동방향

이동방향 (direction)		row 증감	col 증감
북	0	-1	0
북동	1	-1	1
동	2	0	1
남동	3	1	1
남	4	1	0
남서	5	1	-1
서	6	0	-1
북서	7	-1	-1

그림 5.15 이동배열

- 직전 방문점 정보, 직전 이동방향 정보
 - 직전 방문점과 직전 방문점에서의 이동방향 정보 관리 : 스택 이용

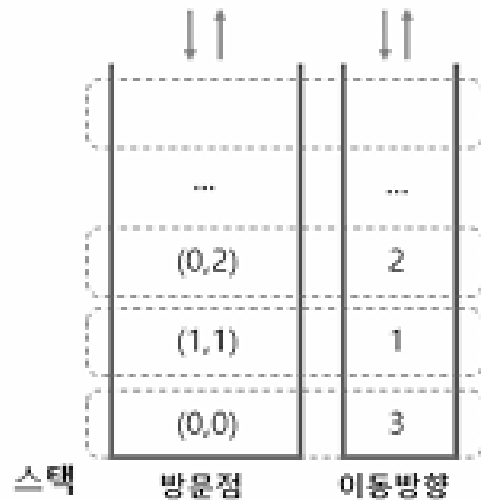


그림 5.16 방문점 스택

방문점 스택 정의

```
struct vertexStack {
    int row;
    int col;
    int dir;
};

struct vertexStack stack[MAX];
int top = -1;
```

- 인접점의 방문 여부 확인
 - mark[M+2][N+2] 배열 정의 : 미로행렬 maze[M+2][N+2]과 같은 크기임
 - 모든 배열원소는 0으로 초기화하고, 방문하면 1로 변경함

코드 5.10 미로 알고리즘

```
1. /* 시작점 초기화 */
   1) 시작점 (row, col)=(1,1), 이동방향 dir=0, found=0 으로 초기화함
   2) 시작점, 이동방향 정보를 방문점, 이동방향 스택에 저장함

2. /* 미로탐색 */
   while (스택≠공집합 and found≠1) { // 미로 미발견
     1) 스택의 원소를 삭제해서 방문위치(row, col) 및 이동방향(dir)을 지정함
     2) while (dir<8 and found≠1) { // 미로 미발견
        nextRow=row+move[dir][0]; nextCol=col+move[dir][1];
        if (nextRow=ExitRow and nextCol=ExitCol) found=1; // 미로발견
        else if (다음 방문점이 통로이고 이미 방문한 점이 아니면) {
          ① 다음 방문점에 방문 표시함;
          ② 이동방향을 1 증가시킨후, 현재 방문위치와 이동방향을 스택에 저장함;
          ③ 다음 방문위치로 이동함(방문위치 및 이동방향 변경)
        }
        else 이동방향을 1 증가시킴;
      }
   }

6. /* 미로탐색 결과 출력 */
   if (found=1) 스택의 방문점들(미로경로)을 순차출력(탐색경로)함;
   else "입구에서 출구로의 경로없음"을 출력함;
```

```

#include<stdio.h>
#include<stdlib.h>
#define MAX 1000
#define M 11
#define N 15
#define ExitRow 12
#define ExitCol 16
#define TRUE 1
#define FALSE 0

typedef struct { int row; int col; int dir; } vertex;
int mark[13][17]; // 모두 0으로 초기화
int maze[13][17] = { ... 미로행렬 생략 ... };
int move[8][2] = { {-1,0}, {-1,1}, {0,1}, {1,1}, {1,0}, {1,-1}, {0,-1}, {-1,-1} };
vertex stack[MAX];
int top = -1;

```

```

void push(vertex c) { ... 코드 생략 .. }
vertex pop() { ... 코드 생략 .. }
int main(void) {
    int i, row, col, nextRow, nextCol, dir;
    int found = FALSE;
    vertex current;

    mark[1][1] = 1; // 방문설정(시작점)
    row=1, col=1, dir=0; current.row=row; current.col=col; current.dir=dir;
    push(current);

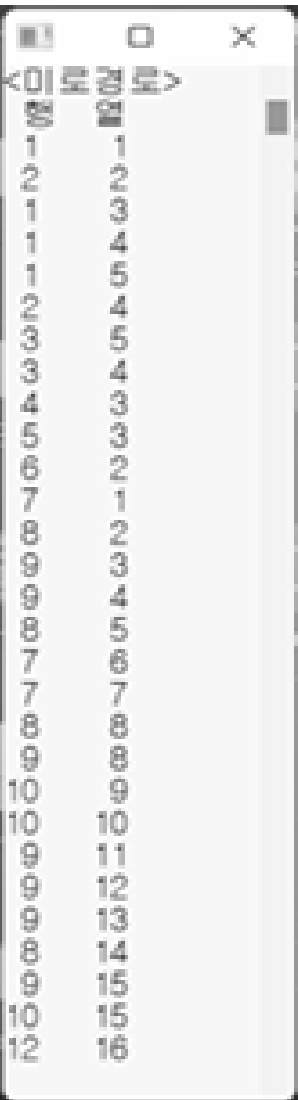
    while(top!=-1 && found==FALSE) { // 방문 안한곳
        current=pop(); row=current.row; col=current.col; dir=current.dir;
        while( dir<8 && found==FALSE ) {
            nextRow = row+move[dir][0]; nextCol=col+move[dir][1];
            if (nextRow==ExitRow && nextCol==ExitCol) { found=TRUE; } // 경로 찾을
            else if (maze[nextRow][nextCol]!=1 && mark[nextRow][nextCol]!=1) {
                mark[nextRow][nextCol] = 1; // 방문표시
                current.row=row; current.col=col; current.dir=dir+1;
                push(current); row=nextRow; col=nextCol; dir= 0;
            } // 방문위치 변경 : 다음 방문점이동
            else dir=dir+1;
        }
    }
    if (found==TRUE) {
        printf("<미로경로>\n"); printf(" 행 열\n");
        for (i=0; i<=top; i++) printf("%2d%5d\n", stack[i].row, stack[i].col);
        printf("%2d%5d\n", ExitRow, ExitCol);
    }
    else printf("경로 없음!!\n");
}

```

그림 5.17 미로찾기 프로그램 실행 결과

[입력 미로배열 예]

```
int maze[13][17] = {
    {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1},
    {1,0,1,0,0,0,1,1,0,0,0,1,1,1,1,1,1},
    {1,1,0,0,0,1,1,0,1,1,1,0,0,1,1,1,1},
    {1,0,1,1,0,0,0,0,1,1,1,1,0,0,1,1,1},
    {1,1,1,0,1,1,1,1,0,1,1,0,1,1,0,0,1},
    {1,1,1,0,1,0,0,1,0,1,1,1,1,1,1,1,1},
    {1,0,0,1,1,0,1,1,1,0,1,0,0,1,0,1,1},
    {1,0,1,1,1,1,0,0,1,1,1,1,1,1,1,1,1},
    {1,0,0,1,1,0,1,1,0,1,1,1,1,1,0,1,1},
    {1,1,1,0,0,0,1,1,0,1,1,0,0,0,0,0,1},
    {1,0,0,1,1,1,1,1,0,0,0,1,1,1,1,0,1},
    {1,0,1,0,0,1,1,1,1,1,0,1,1,1,1,0,1},
    {1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1}
}
```



```
<미로경로>
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
1 0 1 0 0 0 1 1 0 0 0 1 1 1 1 1 1
2 1 1 0 0 0 1 1 0 1 1 1 0 0 1 1 1
3 1 0 1 1 0 0 0 0 1 1 1 1 0 0 1 1
4 1 1 1 0 1 1 1 1 0 1 1 0 1 1 0 0
5 1 1 1 0 1 0 0 1 0 1 1 1 1 1 1 1
6 1 0 0 1 1 0 1 1 1 0 1 0 0 1 0 1
7 1 0 1 1 1 1 0 0 1 1 1 1 1 1 1 1
8 1 0 0 1 1 0 1 1 0 1 1 1 1 1 0 1
9 1 1 1 0 0 0 1 1 0 1 1 0 0 0 0 1
10 1 0 0 1 1 1 1 1 0 0 0 1 1 1 1 0
11 1 0 1 0 0 1 1 1 1 1 0 1 1 1 1 0
12 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
13
14
15
16
```

